12 - Advanced Python - 1

→ Newly added features in python.

- Following are some of the newly added features in python programming language.

→ Walrus Operator :

The walrus operator (:=), introduced in python 3.8, allows you to assign values to variables as part of an expression. This operator, named for its resemblance to the eyes and tusks of a walrus, is officially called the "assignment expression".

1 - 201

using. walrus operator :-

```
if (n := len([1, 2, 3, 4, 5])) > 3:
    print (f"list is long({n}, expected <=3")
```

Output : List is too long (5 elements, expected<=3)

→ in this example, n is assigned the value of len([1, 2, 3, 4, 5]) and then used in the comparison within if statement

Types of Definitions in Python.

Type hints are added using colon (:) syntax for variables and the -> syntax for function return types.

Variables type hint :

```
age : int = 25
```

function type hints.

```
def greeting (name : str) -> str :
    return f"Hello, {name}!"
```

Usage.

```
print ( greeting ("Alice")) # output: hello, Alice!
```

→ Advanced type hints:

Python's typing module provides ~~word~~ more advanced type hints, such as List, Tuples, Dict, and Union.

You can import List, Tuples and Dict types from the typing module like this :

→ from typing import List, Tuple, Dict, Union.

- The Syntax of types looks something like this:

```
from typing import List, Tuple, Dict, Union.

#List of integers.
numbers : List [int] = [1, 2, 3, 4, 5]

#Tuple of string and an integer.
person : Tuple [str, int] = ("Alice", 30)

# Dictionary with string keys and integer values.
Scores : Dict [str, int] = {"Alice":90, "Bob":85}
```

Union type for variables that can hold multiple types.

```
identifier : Union [int, str] = "ID123"
identifier = 12345 # Also valid.
```

→ These annotations help in making the code self-documenting and allow developers to understand the data structures used at a glance.

Match Case:

Python 3.10 introduced the match statement, which is similar to the switch statement found in other languages.

The basic syntax of the match ~~of the~~ statement involves matching a variable against several case using the case keyword.

→
```
def http_status (status):
    match status :
        case 200:
            return "OK"
        case 404:
            return "Not found"
        case 500:
            return "Internal Server Error"
        case _ :
            return "Unknown status"
```

Usage.

```
print (http_status (200)) # output : "OK"
print (http_status (404)) # output : Not found
print (http_status (500)) # Output : ~~Unknown~~ Internal
                                    Server error.
print (http_status (403)) # Output : Unknown status
```

→ Dictionary Merge & Update Operators :

New operator | and |= allow for merging and updating dictionaries.

```
dict 1 = {'a' : 1 , 'b' : 2 }
dict 2 = {'b' : 3 , 'c' : 4}
merge = dict 1 | dict 2.
print (merge) # output : {'a' : 1 , 'b' : 3 , 'c' : 4}
```

→ You can now use multiple content manager in a single with statement more cleanly using the parenthesised content managers.

```
with (
        open ('file 1 . txt') as f1,
        open ('file 2 . txt') as f2,
):
        process files,
```

:→ Exception handling in Python :

There are many build-in exceptions which are raised in python when sometimes goes wrong.

- Exception in python can be handled using a try statement. The code that handles the exception is written in the except clause.

```
try:
    #code which might throw exception
except Exception as e:
    print (e)
```

When the exception is handled, the code flow continues without program interruption.

We can also specify the exception to catch like below:

```
try:
    #code
except ZeroDivisionError:
    #code
except TypeError:
    # Code
except :
    # Code    #All other exceptions are
                handled here.
```

- Raising Exceptions :

We can raise custom exceptions using the 'rise' keyword in python.

- Try with Else Clause :

Sometimes we want to run a piece of code when try was successful.

```
try :
    # code
except :
    # code
else :
    # code        # this is executed only if try was successful.
```

- Try with Finally :
Python offers a 'finally' clause which ensures execution of a piece of code inspective of the exception.

```
try :
    # code
except :
    # code
Finally :
    # code        # Executed regardless of errors.
```

- If __NAME__ == '__MAIN__' IN PYTHON :

'__name__' evaluates to the name of the module in python from where the program is ran.

If the module is being run directly from the command line, the '__name__' is set to

String "__main__". Thus, this behaviour is used to check whether the module is run directly or imported to another file.

The Global Keyword:

'global' keyword is used to modify the variable outside of the current scope.

ENumerate Function in python:

The 'enumerate' function adds counter to an iterable and returns it.

```
For i, item in enumerate(list 1):
    print(i, item)    # Prints the items of
                        list 1 with index.
```

List Comprehensions:

List Comprehensions is an elegant way to create lists based on existing lists.

```
List 1 = [1, 7, 12, 11, 22]
List 2 = [i for item in list 1 if item > 8]
```